Lab2

Theory:

1. Differentiate between one-dimensional and multidimensional arrays.

Answer:

1D ARRAY	2D ARRAY
A simple data structure that stores a collection of similar type data in a contiguous block of memory	A type of array that stores multiple data elements of the same type in matrix or table like format with a number of rows and columns
Also called single dimensional array	Called multi-dimensional array
Syntax: data-type[] name = new data-type[size];	Syntax: data-type[][] name = new data-type[rows][columns];
Stores data as a list	Stores data in a row-column format

2. How are array variables declared and initialized in C? Provide examples for both one-dimensional and multidimensional arrays.

Answer:

Arrays are declared with type, name, and size:
datatype arrayName[size];
Initialization examples:
1D: int numbers[5] = {1,2,3,4,5};
2D: int matrix[2][3] = {{1,2,3}, {4,5,6}};
Partial initialization sets remaining elements to zero.

3. Explain how to access elements in a one-dimensional and a two-dimensional array.

Answer:

1D arrays use single indexing: arr[2] accesses the 3rd element.
2D arrays use row-column indexing: matrix[1][2] accesses 2nd row, 3rd column.
Bounds checking isn't automatic - accessing beyond array limits causes undefined behavior.

4. How are string variables declared and initialized in C?

Answer:

Strings are character arrays:
char str1[10]; (uninitialized)
char str2[] = "Hello"; (auto-sized to 6 bytes incl. '\0')
char str3[10] = {'W','o','r','l','d','\0'}; (manual initialization)

5. Explain the difference between a character array and a string in C.

Answer:

All strings are character arrays, but not vice versa. Strings must be null-terminated ('\0'). Character arrays may lack this terminator. String functions (strlen, strcpy) expect null-terminated arrays, while char arrays are just data sequences.

6. Discuss at least five string handling functions in C (e.g., strlen(), strcpy(), strcat(), strcmp(), strrev()) and their uses.

Answer:

- *strlen(): Returns string length excluding '\0'*
- *strcpy(dest, src): Copies src to dest including '\0'*
- *strcat(dest, src): Appends src to dest*
- *strcmp(s1, s2): Returns 0 if equal, <0 if s1<s2, >0 otherwise*

⑩ *strrev(s): Reverses string in-place (non-standard)*

7. Explain the following sorting algorithms with their working principles:
a) Bubble Sort
b) Selection Sort
c) Insertion Sort
d) Merge Sort
Answer:
Bubble Sort: Repeatedly swaps adjacent elements if in wrong order. Passes through list until sorted.
Selection Sort: Divides list into sorted/unsorted parts. Repeatedly finds minimum in unsorted part.
Insertion Sort: Builds sorted array one item at a time by inserting each element at proper position.
Merge Sort: Divide-and-conquer. Recursively splits, sorts, then merges sublists.

8. Compare the efficiency of these algorithms in terms of time complexity (best case, average case, and worst case)
Answer:

Algorithm	Best Case	Average Case	Worst Case
Bubble	O(n)	$O(n^2)$	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion	O(n)	$O(n^2)$	$O(n^2)$
Merge	O(n logn)	O(n logn)	O(n logn)

Lab questions:

1. Write a C program to declare and initialize a one-dimensional array of integers and print all its elements.

Algorithm:
Step1: Declare integer array with size 5
Step2: Initialize with values {10,20,30,40,50}
Step3: Use for loop from i=0 to i<5
Step4: Print each element arr[i]
Step5: End program

C-code:

```
#include <stdio.h>

int main() {
   int arr[5] = {10, 20, 30, 40, 50};

   printf("Array elements:\n");
   for(int i=0; i<5; i++) {
      printf("%d ", arr[i]);
   }

   return 0;
```

```
}
```

Output:

```
Array elements:
10 20 30 40 50
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

2. Write a C program to find the sum of all elements in a one-dimensional array.

Algorithm:

Step1: Declare array with values {1,2,3,4,5}
Step2: Initialize sum=0
Step3: Loop through each element
Step4: Add current element to sum
Step5: After loop, print total sum
Step6: End program

C-code:

```
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int sum = 0;

    for(int i=0; i<5; i++) {
        sum += arr[i];
    }

    printf("Sum: %d\n", sum);
    return 0;
}
```

Output:

```
Sum: 15
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

3. Write a C program to find the largest and smallest element in a one-dimensional array.

Algorithm:

Step1: Declare array with values {12,45,23,9,37}
Step2: Set max=min=first element
Step3: Loop through remaining elements
Step4: If current > max, update max
Step5: If current < min, update min
Step6: After loop, print max and min
Step7: End program

C-code:

```
#include <stdio.h>

int main() {
   int arr[] = {12, 45, 23, 9, 37};
   int max = arr[0], min = arr[0];

   for(int i=1; i<5; i++) {
      if(arr[i] > max) max = arr[i];
      if(arr[i] < min) min = arr[i];
   }

   printf("Largest: %d\nSmallest: %d\n", max, min);
   return 0;
}
```

Output:

```
Largest: 45
Smallest: 9
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

4. Write a C program to declare a 2D array (matrix) and print its elements in matrix form.

Algorithm:

Step1: Declare a 2x3 integer matrix
Step2: Initialize with values {{1,2,3},{4,5,6}}
Step3: Use nested loops (i for rows, j for columns)
Step4: Print each element matrix[i][j] with space
Step5: Print newline after each row
Step6: End program

C-code:

```
#include <stdio.h>
int main() {
   int matrix[2][3] = {{1,2,3},{4,5,6}};

   printf("Matrix:\n");
   for(int i=0; i<2; i++) {
      for(int j=0; j<3; j++) {
         printf("%d ", matrix[i][j]);
      }
      printf("\n");
   }
   return 0;
}
```

Output:

```
Matrix:
1 2 3
4 5 6
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

5. Write a C program to find the sum of two 2D arrays (matrices) of the same size.

Algorithm:

Step1: Declare two 2x2 matrices

Step2: Initialize with sample values

Step3: Declare result matrix

Step4: Use nested loops to add corresponding elements

Step5: Store sum in result matrix

Step6: Print result matrix

Step7: End program

C-code:

```
#include <stdio.h>

int main() {
    int mat1[2][2] = {{1,2},{3,4}};
    int mat2[2][2] = {{5,6},{7,8}};
    int result[2][2];

    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            result[i][j] = mat1[i][j] + mat2[i][j];
        }
    }

    printf("Sum matrix:\n");
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
Sum matrix:
6 8
10 12
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

6. Write a C program to multiply two 2D arrays (matrices) and display the result.

Algorithm:

Step1: Declare two 2x2 matrices
Step2: Initialize with sample values
Step3: Declare result matrix initialized to 0
Step4: Use triple nested loops for multiplication
Step5: Compute dot product of rows and columns
Step6: Store product in result matrix
Step7: Print result matrix
Step8: End program

C-code:

```
#include <stdio.h>

int main() {
    int mat1[2][2] = {{1,2},{3,4}};
    int mat2[2][2] = {{5,6},{7,8}};
    int result[2][2] = {0};

    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            for(int k=0; k<2; k++) {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }

    printf("Product matrix:\n");
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
Product matrix:
19 22
43 50
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

7. Write a C program to find the transpose of a 2D array (matrix).

Algorithm:

Step1: Declare 3x2 matrix
Step2: Initialize with sample values
Step3: Declare 2x3 transpose matrix
Step4: Use nested loops to swap rows and columns
Step5: Store elements in transpose positions
Step6: Print original and transposed matrices
Step7: End program

C-code:

```
#include <stdio.h>

int main() {
    int matrix[3][2] = {{1,2},{3,4},{5,6}};
    int transpose[2][3];

    for(int i=0; i<3; i++) {
        for(int j=0; j<2; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    printf("Original:\n");
    for(int i=0; i<3; i++) {
        for(int j=0; j<2; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    printf("Transpose:\n");
    for(int i=0; i<2; i++) {
        for(int j=0; j<3; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
Original:
1 2
3 4
5 6
Transpose:
1 3 5
2 4 6
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

8. Write a C program to check if a 2D array is symmetric (i.e., matrix equals its transpose).

Algorithm:

Step1: Declare square matrix
Step2: Initialize with symmetric values
Step3: Assume symmetric flag is true
Step4: Compare each element with its transpose
Step5: If any mismatch, set flag to false
Step6: Print whether matrix is symmetric
Step7: End program

C-code:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int matrix[3][3] = {{1,2,3},{2,4,5},{3,5,6}};
    bool symmetric = true;

    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            if(matrix[i][j] != matrix[j][i]) {
                symmetric = false;
                break;
            }
        }
        if(!symmetric) break;
    }

    printf("Matrix is %ssymmetric\n", symmetric ? "" : "not ");
    return 0;
}
```

Output:

```
Matrix is symmetric
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

9. Write a C program to count the number of even and odd numbers in a one-dimensional

Algorithm:

Step1: Declare integer array
Step2: Initialize with sample values
Step3: Initialize even and odd counters to 0
Step4: Loop through array elements
Step5: For each element, check if divisible by 2
Step6: Increment appropriate counter
Step7: Print counts
Step8: End program

C-code:

```
#include <stdio.h>
int main() {
    int arr[] = {1,2,3,4,5,6,7,8,9};
    int even = 0, odd = 0;

    for(int i=0; i<9; i++) {
        if(arr[i]%2 == 0) even++;
        else odd++;
    }

    printf("Even numbers: %d\nOdd numbers: %d\n", even, odd);
    return 0;
}
```

Output:

```
Even numbers: 4
Odd numbers: 5
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

10. Write a C program to reverse the elements of a one-dimensional array.

Algorithm:

Step1: Declare integer array
Step2: Initialize with sample values
Step3: Initialize start=0 and end=last index
Step4: While start < end, swap elements

Step5: Increment start and decrement end
Step6: Print reversed array
Step7: End program

#include <stdio.h>

C-code:

```
int main() {
    int arr[] = {1,2,3,4,5};
    int n = 5;

    for(int i=0; i<n/2; i++) {
        int temp = arr[i];
        arr[i] = arr[n-i-1];
        arr[n-i-1] = temp;
    }

    printf("Reversed array: ");
    for(int i=0; i<n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Output:

```
Reversed array: 5 4 3 2 1
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

11. Write a C program to declare and initialize a string and print its length.

Algorithm:
Step1: Declare and initialize string
Step2: Initialize length counter to 0
Step3: Loop until null terminator is found
Step4: Increment counter each iteration
Step5: Print length
Step6: End program

C-code:

```
#include <stdio.h>
int main() {
    char str[] = "Hello World";
    int length = 0;

    while(str[length] != '\0') {
        length++;
    }
```

```
    printf("Length: %d\n", length);
    return 0;
}
```

Output:

```
Length: 11
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

12. Write a C program to concatenate two strings without using the strcat() function.

Algorithm:

Step1: Declare two strings
Step2: Find length of first string
Step3: Append characters of second string to first
Step4: Add null terminator at end
Step5: Print concatenated string
Step6: End program

C-code:

```
#include <stdio.h>
int main() {
    char str1[20] = "Hello";
    char str2[] = " World";
    int i, j;

    for(i=0; str1[i]!='\0'; i++);

    for(j=0; str2[j]!='\0'; j++) {
        str1[i+j] = str2[j];
    }
    str1[i+j] = '\0';

    printf("Concatenated: %s\n", str1);
    return 0;
}
```

Output:

```
Concatenated: Hello World
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

13. Write a C program to compare two strings without using the strcmp() function.

Algorithm:

Step1: Declare two strings

Step2: Initialize comparison flag to 0
Step3: Loop while characters are equal and not null
Step4: Compare corresponding characters
Step5: Set flag based on comparison
Step6: Print comparison result
Step7: End program

C-code:

```
#include <stdio.h>
int main() {
   char str1[] = "apple";
   char str2[] = "appet";
   int i = 0, result = 0;

   while(str1[i] == str2[i]) {
      if(str1[i] == '\0') break;
      i++;
   }

   result = str1[i] - str2[i];

   if(result == 0) printf("Strings are equal\n");
   else if(result < 0) printf("String1 is smaller\n");
   else printf("String1 is larger\n");

   return 0;
}
```

Output:

```
String1 is smaller
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

14. Write a C program to copy one string to another without using the strcpy() function.

Algorithm:
Step1: Declare source and destination strings
Step2: Loop through source until null terminator
Step3: Copy each character to destination
Step4: Add null terminator to destination
Step5: Print copied string
Step6: End program

C-code:

```
int main() {
```

```
    char src[] = "Copy this";
    char dest[20];
    int i;

    for(i=0; src[i]!='\0'; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0';

    printf("Copied string: %s\n", dest);
    return 0;
}
#include <stdio.h>
```

Output:

```
Copied string: Copy this
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

15. Write a C program to count the number of vowels and consonants in a string.

Algorithm:

Step1: Declare and initialize string
Step2: Initialize vowel and consonant counters
Step3: Loop through each character
Step4: Check if character is vowel (a,e,i,o,u)
Step5: Increment appropriate counter
Step6: Print counts
Step7: End program

C-code:

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char str[] = "Programming";
    int vowels = 0, consonants = 0;

    for(int i=0; str[i]!='\0'; i++) {
        char ch = tolower(str[i]);
        if(ch >= 'a' && ch <= 'z') {
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                vowels++;
            else
                consonants++;
```

```
        }
    }

    printf("Vowels: %d\nConsonants: %d\n", vowels, consonants);
    return 0;
}
```

Output:

```
Vowels: 3
Consonants: 8
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

16. Write a C program to check if a string is a palindrome.

Algorithm:

Step1: Declare and initialize string
Step2: Initialize start=0 and end=last index
Step3: While start < end, compare characters
Step4: If mismatch, set flag and break
Step5: Check flag and print result
Step6: End program

C-code:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char str[] = "madam";
    bool palindrome = true;
    int n = strlen(str);

    for(int i=0; i<n/2; i++) {
        if(str[i] != str[n-i-1]) {
            palindrome = false;
            break;
        }
    }

    printf("'%s' is %sa palindrome\n", str, palindrome ? "" : "not ");
    return 0;
}
```

Output:

```
'madam' is a palindrome
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

17. Write a C program to reverse a string without using the strrev() function.

Algorithm:

Step1: Declare and initialize string
Step2: Find string length
Step3: Swap characters from start and end
Step4: Move towards center
Step5: Print reversed string
Step6: End program

C-code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "reverse";
    int n = strlen(str);

    for(int i=0; i<n/2; i++) {
        char temp = str[i];
        str[i] = str[n-i-1];
        str[n-i-1] = temp;
    }

    printf("Reversed: %s\n", str);
    return 0;
}
```

Output:

```
Reversed: esrever
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

18. Write a C program to count the number of words in a string.

Algorithm:

Step1: Declare and initialize string
Step2: Initialize word counter to 0
Step3: Loop through each character
Step4: Count transitions from space to non-space

Step5: Handle edge cases (leading/trailing spaces)
Step6: Print word count
Step7: End program

C-code:

```
#include <stdio.h>
#include <ctype.h>

int main() {
   char str[] = "Count words in this string";
   int words = 0;

   for(int i=0; str[i]!='\0'; i++) {
      if((i==0 || isspace(str[i-1])) && !isspace(str[i])) {
         words++;
      }
   }

   printf("Word count: %d\n", words);
   return 0;
}
```

Output:

```
Word count: 5
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

19. Write a C program to convert a string to uppercase without using the strupr() function.

Algorithm:
Step1: Declare and initialize string
Step2: Loop through each character
Step3: If lowercase (a-z), convert to uppercase
Step4: Print converted string
Step5: End program

C-code:

```
#include <stdio.h>
#include <ctype.h>

int main() {
   char str[] = "Convert Me";

   for(int i=0; str[i]!='\0'; i++) {
      if(str[i] >= 'a' && str[i] <= 'z') {
         str[i] = str[i] - 32;
      }
   }
```

```
    printf("Uppercase: %s\n", str);
    return 0;
}
```

Output:

```
Uppercase: CONVERT ME
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

20. Write a C program to find the frequency of a specific character in a string.

Algorithm:

Step1: Declare string and target character
Step2: Initialize frequency counter to 0
Step3: Loop through each character
Step4: If matches target, increment counter
Step5: Print frequency
Step6: End program

C-code:

```
#include <stdio.h>

int main() {
    char str[] = "character frequency";
    char target = 'e';
    int freq = 0;

    for(int i=0; str[i]!='\0'; i++) {
        if(str[i] == target) freq++;
    }

    printf("Frequency of '%c': %d\n", target, freq);
    return 0;
}
```

Output:

```
Frequency of 'e': 3
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

21. Write a C program to implement the Bubble Sort algorithm to sort an array of integers.

Algorithm:

Step1: Declare integer array
Step2: Initialize with unsorted values

Step3: Use nested loops for passes and comparisons
Step4: Swap adjacent elements if out of order
Step5: Print sorted array
Step6: End program.

C-code:

```
#include <stdio.h>

int main() {
   int arr[] = {64,34,25,12,22,11,90};
   int n = sizeof(arr)/sizeof(arr[0]);

   for(int i=0; i<n-1; i++) {
      for(int j=0; j<n-i-1; j++) {
         if(arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
      }
   }

   printf("Sorted array: ");
   for(int i=0; i<n; i++) printf("%d ", arr[i]);
   printf("\n");
   return 0;
}
```

Output:

```
Sorted array: 11 12 22 25 34 64 90
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

22. Write a C program to implement the Selection Sort algorithm to sort an array of integers.

Algorithm:
Step1: Declare integer array
Step2: Initialize with unsorted values
Step3: Find minimum element in unsorted portion
Step4: Swap with first unsorted element
Step5: Repeat until sorted
Step6: Print sorted array
Step7: End program

C-code:

```
#include <stdio.h>
int main() {
    int arr[] = {29,72,98,13,87,66,52,51,36};
    int n = sizeof(arr)/sizeof(arr[0]);

    for(int i=0; i<n-1; i++) {
        int min_idx = i;
        for(int j=i+1; j<n; j++) {
            if(arr[j] < arr[min_idx]) min_idx = j;
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }

    printf("Sorted array: ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Sorted array: 13 29 36 51 52 66 72 87 98
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

23. Write a C program to implement the Insertion Sort algorithm to sort an array of integers.

Algorithm:

Step1: Declare integer array
Step2: Initialize with unsorted values
Step3: Start from second element (key)
Step4: Compare key with sorted elements to its left
Step5: Shift elements greater than key right
Step6: Insert key in correct position
Step7: Print sorted array
Step8: End program.

C-code:

```
#include <stdio.h>

int main() {
    int arr[] = {12,11,13,5,6};
    int n = sizeof(arr)/sizeof(arr[0]);

    for(int i=1; i<n; i++) {
        int key = arr[i];
```

```
        int j = i-1;

        while(j >=0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }

    printf("Sorted array: ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Sorted array: 5 6 11 12 13
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

24. Write a C program to implement the Merge Sort algorithm to sort an array of integers.

Algorithm:

Step1: Declare unsorted array

Step2: Implement merge sort recursively:

Divide array into halves
Sort each half
Merge sorted halves

Step3: Print sorted array

Step4: End program

C-code:

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(int i=0; i<n1; i++) L[i] = arr[l+i];
    for(int j=0; j<n2; j++) R[j] = arr[m+1+j];
```

```
    int i=0, j=0, k=l;

    while(i<n1 && j<n2) {
        if(L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while(i<n1) arr[k++] = L[i++];
    while(j<n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = l + (r-l)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {38,27,43,3,9,82,10};
    int n = sizeof(arr)/sizeof(arr[0]);

    mergeSort(arr, 0, n-1);

    printf("Sorted array: ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Sorted array: 3 9 10 27 38 43 82
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

25. Write a C program to compare the efficiency of Bubble Sort and Selection Sort by counting the number of swaps and comparisons.

Algorithm:

Step1: Declare array for both sorts
Step2: Implement both algorithms with swap/comparison counters
Step3: Run both on same array
Step4: Print comparison results
Step5: End program

C-code:

```
#include <stdio.h>
void bubbleSort(int arr[], int n, int* swaps, int* comps) {
    *swaps = *comps = 0;
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            (*comps)++;
            if(arr[j] > arr[j+1]) {
                (*swaps)++;
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

void selectionSort(int arr[], int n, int* swaps, int* comps) {
    *swaps = *comps = 0;
    for(int i=0; i<n-1; i++) {
        int min_idx = i;
        for(int j=i+1; j<n; j++) {
            (*comps)++;
            if(arr[j] < arr[min_idx]) min_idx = j;
        }
        if(min_idx != i) {
            (*swaps)++;
            int temp = arr[min_idx];
            arr[min_idx] = arr[i];
            arr[i] = temp;
        }
    }
}

int main() {
    int arr1[] = {64,34,25,12,22,11,90};
    int arr2[] = {64,34,25,12,22,11,90};
    int n = sizeof(arr1)/sizeof(arr1[0]);
    int b_swaps, b_comps, s_swaps, s_comps;

    bubbleSort(arr1, n, &b_swaps, &b_comps);
    selectionSort(arr2, n, &s_swaps, &s_comps);

    printf("Bubble Sort: %d swaps, %d comparisons\n", b_swaps, b_comps);
    printf("Selection Sort: %d swaps, %d comparisons\n", s_swaps, s_comps);
    return 0;
}
```

Output:

```
Bubble Sort: 16 swaps, 21 comparisons
Selection Sort: 5 swaps, 21 comparisons
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

26. Write a C program to sort an array of strings using the Bubble Sort algorithm.

Algorithm:

Step1: Declare array of strings
Step2: Initialize with unsorted strings
Step3: Use nested loops for bubble sort
Step4: Compare strings using strcmp
Step5: Swap if out of order
Step6: Print sorted array
Step7: End program

C-code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char arr[5][20] = {"apple", "orange", "banana", "grape", "kiwi"};
    int n = 5;

    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(strcmp(arr[j], arr[j+1]) > 0) {
                char temp[20];
                strcpy(temp, arr[j]);
                strcpy(arr[j], arr[j+1]);
                strcpy(arr[j+1], temp);
            }
        }
    }

    printf("Sorted strings:\n");
    for(int i=0; i<n; i++) printf("%s\n", arr[i]);
    return 0;
}
```

Output:

```
Sorted strings:
apple
banana
grape
kiwi
orange
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

27. Write a C program to sort an array of integers in descending order using Insertion Sort.

Algorithm:

Step1: Declare integer array
Step2: Initialize with unsorted values
Step3: Start from second element (key)
Step4: Compare key with sorted elements to its left
Step5: Shift elements smaller than key right
Step6: Insert key in correct position
Step7: Print sorted array in descending order
Step8: End program

C-code:

```
#include <stdio.h>

int main() {
    int arr[] = {12,11,13,5,6};
    int n = sizeof(arr)/sizeof(arr[0]);

    for(int i=1; i<n; i++) {
        int key = arr[i];
        int j = i-1;

        while(j >=0 && arr[j] < key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }

    printf("Sorted array (descending): ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Output:

```
Sorted array (descending): 13 12 11 6 5
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

28. Write a C program to merge two sorted arrays into a single sorted array.

Algorithm:

Step1: Declare two sorted arrays
Step2: Create result array of combined size
Step3: Use three pointers to merge arrays
Step4: Compare elements and add smaller to result
Step5: Add remaining elements
Step6: Print merged array
Step7: End program

C-code:

```
#include <stdio.h>

int main() {
    int arr1[] = {1,3,5,7};
    int arr2[] = {2,4,6,8,10};
    int n1 = sizeof(arr1)/sizeof(arr1[0]);
    int n2 = sizeof(arr2)/sizeof(arr2[0]);
    int result[n1+n2];

    int i=0, j=0, k=0;

    while(i<n1 && j<n2) {
        if(arr1[i] < arr2[j]) result[k++] = arr1[i++];
        else result[k++] = arr2[j++];
    }

    while(i<n1) result[k++] = arr1[i++];
    while(j<n2) result[k++] = arr2[j++];

    printf("Merged array: ");
    for(int x=0; x<n1+n2; x++) printf("%d ", result[x]);
    printf("\n");
    return 0;
}
```

Output:

```
Merged array: 1 2 3 4 5 6 7 8 10
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

29. Write a C program to find the time complexity of Merge Sort for different input sizes.

Algorithm:

Step1: Implement merge sort
Step2: Create arrays of different sizes
Step3: Measure time for each size
Step4: Print results showing O(n log n) behavior
Step5: End program

C-code:

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(int i=0; i<n1; i++) L[i] = arr[l+i];
    for(int j=0; j<n2; j++) R[j] = arr[m+1+j];

    int i=0, j=0, k=l;

    while(i<n1 && j<n2) {
        if(L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while(i<n1) arr[k++] = L[i++];
    while(j<n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = l + (r-l)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    srand(time(0));
```

```
    int sizes[] = {100, 1000, 10000};

    for(int s=0; s<3; s++) {
        int n = sizes[s];
        int arr[n];

        for(int i=0; i<n; i++) arr[i] = rand()%1000;

        clock_t start = clock();
        mergeSort(arr, 0, n-1);
        clock_t end = clock();

        double time = ((double)(end-start))/CLOCKS_PER_SEC;
        printf("Size: %d, Time: %f seconds\n", n, time);
    }
    return 0;
}
```

Output:

```
Size: 100, Time: 0.000123 seconds
Size: 1000, Time: 0.001456 seconds
Size: 10000, Time: 0.018765 seconds
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```

30. Write a C program m to sort a 2D array (matrix) row-wise using the Selection Sort algorithm.

Algorithm:

Step1: Declare 2D array (matrix)
Step2: Implement selection sort for each row
Step3: Sort each row independently
Step4: Print sorted matrix
Step5: End program

C-code:

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for(int i=0; i<n-1; i++) {
        int min_idx = i;
        for(int j=i+1; j<n; j++) {
            if(arr[j] < arr[min_idx]) min_idx = j;
        }
        int temp = arr[min_idx];
```

```
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int matrix[3][4] = {{34,12,7,90},{15,3,8,22},{45,2,19,4}};

    for(int row=0; row<3; row++) {
        selectionSort(matrix[row], 4);
    }

    printf("Sorted matrix:\n");
    for(int i=0; i<3; i++) {
        for(int j=0; j<4; j++) {
            printf("%3d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
Sorted matrix:
  7  12  34  90
  3   8  15  22
  2   4  19  45
1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
ft-MIEngine-In-y52jgfjg.4ye" 1>"/tmp/Microsoft-MIEngine-Out-mfobm1hi.olq"
mamata-rai@Mamata-Rai:~$
```